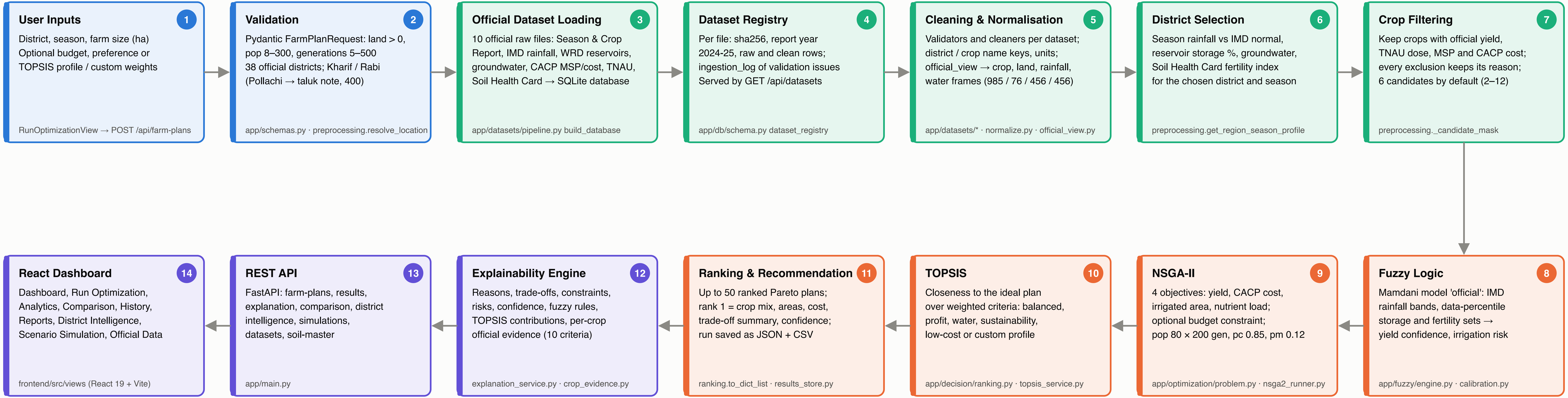
AgroOptimize: proposed methodology (as implemented)

From user input to an explainable crop-allocation recommendation on official Tamil Nadu data



User Inputs 1
District, season, farm size (ha)
Optional budget, preference or
TOPSIS profile / custom weights
RunOptimizationView → POST /api/farm-plans

Validation 2
Pydantic FarmPlanRequest: land > 0,
pop 8–300, generations 5–500
38 official districts; Kharif / Rabi
(Pollachi → taluk note, 400)
app/schemas.py · preprocessing.resolve_location

Official Dataset Loading 3
10 official raw files: Season & Crop
Report, IMD rainfall, WRD reservoirs,
groundwater, CACP MSP/cost, TNAU,
Soil Health Card → SQLite database
app/datasets/pipeline.py build_database

Dataset Registry 4
Per file: sha256, report year
2024-25, raw and clean rows;
ingestion_log of validation issues
Served by GET /api/datasets
app/db/schema.py dataset_registry

Cleaning & Normalisation 5
Validators and cleaners per dataset;
district / crop name keys, units;
official_view → crop, land, rainfall,
water frames (985 / 76 / 456 / 456)
app/datasets/* · normalize.py · official_view.py

District Selection 6
Season rainfall vs IMD normal,
reservoir storage %, groundwater,
Soil Health Card fertility index
for the chosen district and season
preprocessing.get_region_season_profile

Crop Filtering 7
Keep crops with official yield,
TNAU dose, MSP and CACP cost;
every exclusion keeps its reason;
6 candidates by default (2–12)
preprocessing._candidate_mask

Fuzzy Logic 8
Mamdani model 'official': IMD
rainfall bands, data-percentile
storage and fertility sets →
yield confidence, irrigation risk
app/fuzzy/engine.py · calibration.py

NSGA-II 9
4 objectives: yield, CACP cost,
irrigated area, nutrient load;
optional budget constraint;
pop 80 × 200 gen, pc 0.85, pm 0.12
app/optimization/problem.py · nsga2_runner.py

TOPSIS 10
Closeness to the ideal plan
over weighted criteria: balanced,
profit, water, sustainability,
low-cost or custom profile
app/decision/ranking.py · topsis_service.py

Ranking & Recommendation 11
Up to 50 ranked Pareto plans;
rank 1 = crop mix, areas, cost,
trade-off summary, confidence;
run saved as JSON + CSV
ranking.to_dict_list · results_store.py

Explainability Engine 12
Reasons, trade-offs, constraints,
risks, confidence, fuzzy rules,
TOPSIS contributions, per-crop
official evidence (10 criteria)
explanation_service.py · crop_evidence.py

REST API 13
FastAPI: farm-plans, results,
explanation, comparison, district
intelligence, simulations,
datasets, soil-master
app/main.py

React Dashboard 14
Dashboard, Run Optimization,
Analytics, Comparison, History,
Reports, District Intelligence,
Scenario Simulation, Official Data
frontend/src/views (React 19 + Vite)

🟦 Input and validation
🟩 Official data layer
🟧 Decision engine
🟪 Explanation and delivery

Datasets are validated, cleaned and imported once (scripts/preprocess_all.py, also at Docker build); each request reads them through app/data/loader.py (cached per process).
The fuzzy stage runs only where an official reservoir reading exists for the district and season; otherwise it is skipped and flagged (yield left unadjusted).
Features without an official dataset (e.g. live weather) return HTTP 501 instead of estimated values.

Source: AgroOptimize backend (app/), frontend (frontend/src) and docs/data_architecture, branch feature/official-data-migration